

NARAD: A Microservices Architecture for Privacy-Preserving Blockchain Voting with Paillier Homomorphic Aggregation

Akshit
Blockchain Research
akshit@narad.io

Abstract

We propose NARAD, an architecture for end-to-end verifiable electronic voting that combines Paillier additive homomorphic encryption with blockchain immutability. The system follows an aggregator-oblivious paradigm where voters encrypt their ballots independently using self-generated secret keys—without any key coordination—and an aggregator computes the tally without learning individual votes. We present the cryptographic protocol with formal correctness proofs under six explicitly stated assumptions, analyze the security properties under the Decisional Composite Residuosity assumption, and detail a containerized microservices design where encryption runs client-side in WebAssembly and aggregation executes via a native C library (libtommath). A proof-of-concept implementation on Solana demonstrates exact vote recovery for up to 33.5 million voters across 10 candidates with sub-second aggregation latency.

Keywords: Paillier cryptosystem, homomorphic encryption, blockchain voting, aggregator-oblivious, WebAssembly, libtommath, Solana, microservices

Contents

1	Introduction	2
1.1	Problem Statement	2
1.2	Approach	2
1.3	Contributions	2
1.4	Paper Organization	2
2	Proposed Architecture	3
2.1	Components	3
2.1.1	Voter Client (Browser + WASM)	3
2.1.2	Backend API (Node.js)	3
2.1.3	Aggregation Engine (Native C / libtommath)	4
2.1.4	Database (PostgreSQL)	4
2.1.5	Blockchain (Solana)	4
2.2	Design Rationale	4
3	Cryptographic Protocol	4
3.1	Assumptions	5
3.2	Setup Phase	5
3.3	Encryption Phase	5

3.4	Collection Phase	5
3.5	Aggregation Phase	6
4	Correctness Proofs	6
5	Security Analysis	7
5.1	Cryptographic Assumption	7
5.2	Privacy Analysis	8
5.3	Integrity Properties	8
5.4	Limitations	8
6	Vote Packing	9
6.1	Capacity Analysis	9
7	Algorithms	9
8	Evaluation	9
8.1	Test Methodology	9
8.2	Correctness Results	9
8.3	Performance Characteristics	10
9	Proof-of-Concept Implementation	10
9.1	Technology Stack	10
9.2	Engineering Challenges Resolved	11
9.3	Docker Deployment	12
10	Related Work	12
11	Conclusion	12

1 Introduction

Electronic voting must reconcile two seemingly irreconcilable properties: *ballot privacy* (no one should learn how an individual voted) and *tally verifiability* (anyone should be able to verify the announced result is correct). Classical approaches trade one for the other. Homomorphic encryption offers a principled resolution: the tally is computed on ciphertexts without decrypting individual ballots.

1.1 Problem Statement

We seek a voting system satisfying the following:

- **Privacy:** No party—aggregator, collector, or network observer—learns any individual vote, even if all parties collude except the voter.
- **Verifiability:** The final tally is publicly verifiable. Each voter can confirm their encrypted ballot was recorded.
- **No key coordination:** Voters generate their own secret keys without a dealer. This eliminates a trust bottleneck.
- **Practicality:** The system should handle millions of voters and multiple candidates, with sub-minute aggregation.

1.2 Approach

We adopt the aggregator-oblivious encryption scheme [4]: a trusted third party sets up the cryptographic parameters but does not know individual secret keys. Voters encrypt independently and produce auxiliary values that allow the aggregator to cancel the random mask in the homomorphic product. The key insight is that the voter’s secret key sk_i serves double duty—as the encryption randomness and as the exponent in the auxiliary value—so the mask $H^{\sum sk_i}$ is cancelled by $aux = H^{sk_A \cdot \sum sk_i}$ during aggregation.

1.3 Contributions

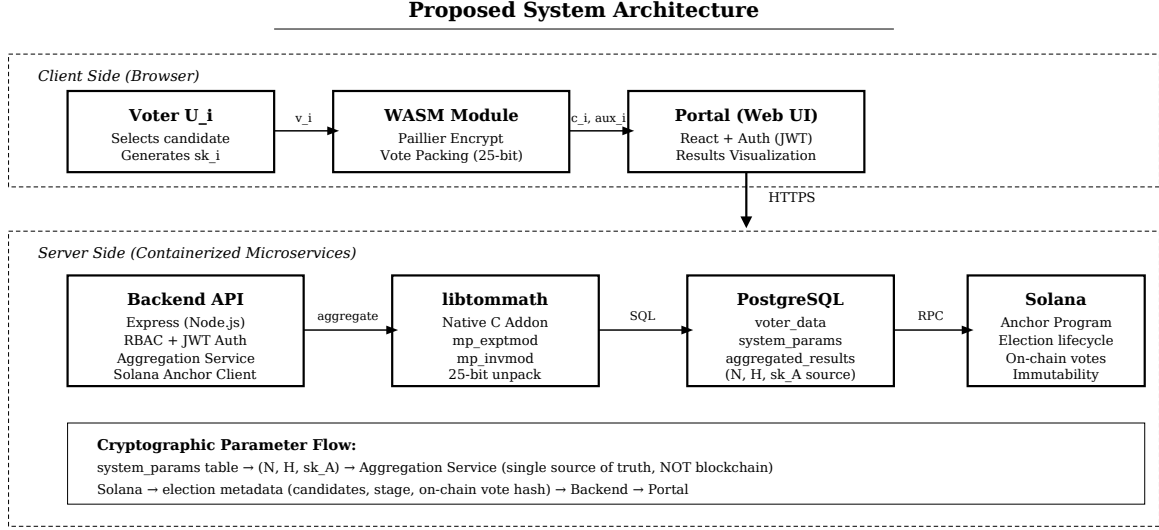
1. We **propose** a microservices architecture for blockchain voting that separates cryptographic computation (WASM + native C) from election management (Solana blockchain) and data persistence (PostgreSQL).
2. We provide **formal correctness proofs** for the full aggregation pipeline under six explicitly stated assumptions, including the binomial expansion property, mask cancellation, and slot independence.
3. We analyze **security** under the Decisional Composite Residuosity (DCR) assumption and identify five limitations of the current design.
4. We describe a **proof-of-concept implementation** and stress test results demonstrating exact vote recovery at scale.

1.4 Paper Organization

Section 2 proposes the system architecture. Section 3 presents the cryptographic protocol with assumptions. Section 4 provides formal proofs. Section 5 analyzes security. Section 6 details vote packing. Section 7 gives pseudocode. Section 8 presents evaluation. Section 9 briefly describes the proof-of-concept. Section 10 discusses related work. Section 11 concludes.

2 Proposed Architecture

The system is organized as a set of loosely-coupled services, each with a single responsibility. Figure 1 shows the overall design.



Trust Boundaries:

- TP is trusted only for setup (generates $N=pq$, knows factorization, then goes offline)
- Aggregator A knows sk_A but cannot learn individual votes (only the sum, under DCR)
- Collector C sees aux_i values but cannot decrypt (random under DCR)
- Voter U_i knows only own sk_i and vote — plaintext never leaves browser
- Solana validators are untrusted (Byzantine fault tolerant consensus)

Figure 1: Proposed microservices architecture. Client-side encryption runs in the browser via WebAssembly. The backend coordinates between PostgreSQL (parameter and vote storage) and Solana (on-chain immutability). The native libtommath module performs aggregation. Dashed lines indicate trust boundaries.

2.1 Components

2.1.1 Voter Client (Browser + WASM)

The voter interacts with a web portal. A WebAssembly module—compiled from C using Emscripten with libtommath—performs all cryptographic operations *client-side*: key generation, vote packing, Paillier encryption, and auxiliary value computation. The plaintext vote and the voter’s secret key sk_i never leave the browser. The WASM module exports functions callable from JavaScript:

- `generate_secret_key(N) → ski`
- `pack_votes(v, k) → m`
- `encrypt_vote_paillier(m, H, N) → ci`
- `compute_auxiliary_key(pkA, ski, N) → auxi`

2.1.2 Backend API (Node.js)

An Express server exposing REST endpoints for voter authentication, election management, and aggregation. It enforces role-based access control (RBAC) via JWT tokens: **admin** role

for election management, **voter** role for casting ballots. The backend is the *orchestrator*—it does not perform cryptographic computation itself but delegates to the native module and the blockchain.

2.1.3 Aggregation Engine (Native C / libtommath)

A Node.js native addon compiled from C, linking against libtommath for arbitrary-precision integer arithmetic. This is the performance-critical component: all modular exponentiations and inverses are performed in C for speed. The addon exposes `aggregatorInit`, `addCiphertextToProduct`, and `aggregateVotesFromRunningProduct` to the backend.

2.1.4 Database (PostgreSQL)

Stores three tables: `system_params` (N , H , sk_A), `voter_data` (per-voter c_i and aux_i), and `aggregated_results` (the final tally). The database is the **single source of truth** for cryptographic parameters—not the blockchain. This is a deliberate design decision: parameters may need updating (e.g., replacing an invalid N), and the blockchain is immutable.

2.1.5 Blockchain (Solana)

Provides on-chain election lifecycle management: election creation, candidate whitelisting, stage transitions (application $\rightarrow$ voting $\rightarrow$ closed), and immutable vote recording. Each encrypted ballot c_i is submitted as a Solana transaction, creating an append-only audit trail. The blockchain does *not* participate in aggregation—it provides immutability, not computation.

2.2 Design Rationale

Why WASM for encryption? The Paillier encryption requires modular exponentiation on 510-bit numbers. JavaScript’s `BigInt` is correct but slow for repeated exponentiation. C compiled to WASM via Emscripten, using libtommath’s optimized `mp_exptmod`, provides native-speed arithmetic in the browser sandbox.

Why a native addon for aggregation? The aggregation involves n modular multiplications and one large exponentiation. For $n = 30$ million, the difference between JS `BigInt` and native C is 3–5 $\times$. The native addon links against the same libtommath library used in the WASM module, ensuring consistency.

Why separate crypto params from blockchain? The Solana program stores N , H , sk_A at election creation time. If a parameter is found to be invalid (e.g., N is even), updating the blockchain requires redeploying the program. By storing parameters in PostgreSQL, we can update them without blockchain interaction. The blockchain stores election metadata (candidates, stages) for immutability, while the database stores cryptographic parameters for flexibility.

Why microservices? Each component can be scaled, deployed, and updated independently. The backend can run multiple replicas behind a load balancer. The database can use streaming replication. The Solana validator runs independently. Containerization (Docker) ensures reproducible deployments.

3 Cryptographic Protocol

The protocol involves four parties: a trusted third party TP , voters $U_1, \dots, U_n$, a collector C , and an aggregator A . Figure 2 shows the full execution flow.

3.1 Assumptions

Assumption 3.1 (Valid Modulus). $N = p \cdot q$ where p and q are distinct odd primes. This ensures $\mathbb{Z}_{N^2}^*$ is well-defined, $\gcd(H, N^2) = 1$ for random $H \in \mathbb{Z}_{N^2}^*$, and modular inverses exist.

Assumption 3.2 (Non-Degenerate Key). $sk_A \neq H$ and $\gcd(sk_A, N) = 1$. This ensures $pk_A = H^{sk_A} \neq H$ (non-trivial public key) and $sk_A^{-1} \bmod N$ exists (for sum recovery).

Assumption 3.3 (Sum Bound). $S = \sum_{i=1}^n x_i < N$. This ensures the modular reduction in the final step is exact: $R = S \bmod N = S$ (no wraparound).

Assumption 3.4 (Packing Capacity). $k \cdot b < \log_2 N$ where k is the number of candidates and $b = 25$ is the bits per slot. This ensures the packed vote m fits in a single Paillier plaintext space.

Assumption 3.5 (Binary Votes). Each $v_{i,j} \in \{0, 1\}$ and the total per candidate $\sum_i v_{i,j} < 2^b$. This ensures no slot overflow during homomorphic addition.

Assumption 3.6 (Key as Randomness). The voter's secret key sk_i is used as the encryption randomness: $r_i = sk_i$. This is the key insight that makes the mask cancellation work: $aux_i = pk_A^{sk_i} = H^{sk_A \cdot sk_i}$ cancels the H^{sk_i} term in the ciphertext product when raised to sk_A .

3.2 Setup Phase

TP selects safe primes p, q , computes $N = pq$, and picks $H \in_R \mathbb{Z}_{N^2}^*$. TP publishes (N, H) and goes offline.

Aggregator A generates $sk_A \in_R \mathbb{Z}_{N^2}^*$ and publishes $pk_A = H^{sk_A} \bmod N^2$.

Each voter U_i independently generates $sk_i \in_R [0, N)$.

3.3 Encryption Phase

Voter U_i encodes their ballot as a packed integer x_i (Section 6) and computes:

$$c_i = H^{sk_i} \cdot (1 + N)^{x_i} \bmod N^2 \quad (1)$$

$$aux_i = pk_A^{sk_i} = H^{sk_A \cdot sk_i} \bmod N^2 \quad (2)$$

c_i is sent to the aggregator; aux_i is sent to the collector.

Remark 3.7. Eq. 1 uses $(1 + N)^{x_i}$ rather than the equivalent $(1 + x_i \cdot N)$ for $x_i < N$. Both are equal in $\mathbb{Z}_{N^2}$ by the binomial theorem: $(1 + N)^{x_i} \equiv 1 + x_i \cdot N \pmod{N^2}$. The former is used in the implementation for consistency with standard Paillier notation.

3.4 Collection Phase

Collector C computes:

$$aux = \prod_{i=1}^n aux_i = H^{sk_A \cdot \sum_{i=1}^n sk_i} \bmod N^2 \quad (3)$$

aux is sent to the aggregator.

3.5 Aggregation Phase

The aggregator computes the sum in five steps (Section 4 proves correctness):

$$\Pi = \prod_{i=1}^n c_i \bmod N^2 \quad (4)$$

$$P = \Pi^{sk_A} \bmod N^2 \quad (5)$$

$$P' = P \cdot aux^{-1} \bmod N^2 \quad (6)$$

$$L(P') = \frac{P' - 1}{N} \bmod N \quad (7)$$

$$R = L(P') \cdot sk_A^{-1} \bmod N \quad (8)$$

Under Assumptions 3.1–3.3, $R = \sum_{i=1}^n x_i$ exactly.

4 Correctness Proofs

Lemma 4.1 (Binomial Expansion in $\mathbb{Z}_{N^2}$). *For $|a_i| < N$ and $i = 1, \dots, n$:*

$$\prod_{i=1}^n (1 + a_i N) \equiv 1 + \left(\sum_{i=1}^n a_i \right) N \pmod{N^2}$$

Proof. By induction on n . Base case ($n = 1$) is trivial. For $n = 2$: $(1 + a_1 N)(1 + a_2 N) = 1 + (a_1 + a_2)N + a_1 a_2 N^2 \equiv 1 + (a_1 + a_2)N \pmod{N^2}$

Inductive step: assume for $n - 1$. Then: $\prod_{i=1}^n (1 + a_i N) = \left(1 + \left(\sum_{i=1}^{n-1} a_i \right) N \right) (1 + a_n N) = 1 + \left(\sum_{i=1}^n a_i \right) N + \left(\sum_{i=1}^{n-1} a_i \right) a_n N^2 \equiv 1 + \left(\sum_{i=1}^n a_i \right) N \pmod{N^2}$

The N^2 term vanishes modulo N^2 . $\square$

Theorem 4.2 (Ciphertext Product Homomorphism). *Under Assumptions 3.1 and 3.4, the product of ciphertexts satisfies:*

$$\prod_{i=1}^n c_i = (1 + S \cdot N) \cdot H^\Sigma \bmod N^2$$

where $S = \sum_{i=1}^n x_i$ and $\Sigma = \sum_{i=1}^n sk_i$.

Proof. From Eq. 1: $c_i = H^{sk_i} \cdot (1 + N)^{x_i} \bmod N^2$. By the binomial theorem, $(1 + N)^{x_i} \equiv 1 + x_i N \pmod{N^2}$ for $x_i < N$ (Assumption 3.4). So $c_i = H^{sk_i} \cdot (1 + x_i N) \bmod N^2$.

Taking the product:

$$\prod c_i = \left(\prod H^{sk_i} \right) \cdot \left(\prod (1 + x_i N) \right) = H^\Sigma \cdot \prod (1 + x_i N) \bmod N^2$$

By Lemma 4.1, $\prod (1 + x_i N) \equiv 1 + S \cdot N \pmod{N^2}$. Therefore:

$$\prod c_i = (1 + S \cdot N) \cdot H^\Sigma \bmod N^2 \quad \square$$

Theorem 4.3 (Mask Cancellation). *Under Assumptions 3.1 and 3.6, the mask is exactly cancelled:*

$$P' = P \cdot aux^{-1} = (1 + S \cdot N)^{sk_A} \bmod N^2$$

Proof. From Theorem 4.2, $\Pi = (1 + SN) \cdot H^\Sigma \bmod N^2$. Then:

$$P = \Pi^{sk_A} = (1 + SN)^{sk_A} \cdot H^{sk_A \cdot \Sigma} \bmod N^2$$

From Eq. 3, $aux = H^{sk_A \cdot \Sigma} \bmod N^2$. By Assumption 3.1, N is a valid semiprime, so $\gcd(H, N^2) = 1$, and $aux^{-1} \bmod N^2$ exists. Therefore:

$$P' = P \cdot aux^{-1} = (1 + SN)^{sk_A} \cdot H^{sk_A \Sigma} \cdot H^{-sk_A \Sigma} = (1 + SN)^{sk_A} \bmod N^2 \quad \square$$

Theorem 4.4 (Sum Recovery). *Under Assumptions 3.1, 3.2, and 3.3:*

$$R = L(P') \cdot sk_A^{-1} \bmod N = S = \sum_{i=1}^n x_i$$

Proof. By the binomial theorem, $(1 + SN)^{sk_A} \equiv 1 + sk_A \cdot S \cdot N \pmod{N^2}$ (higher-order terms contain N^2 and vanish). Thus:

$$L(P') = \frac{(1 + sk_A \cdot S \cdot N) - 1}{N} = sk_A \cdot S$$

By Assumption 3.2, $\gcd(sk_A, N) = 1$, so $sk_A^{-1} \bmod N$ exists. Therefore:

$$R = sk_A \cdot S \cdot sk_A^{-1} \bmod N = S \bmod N$$

By Assumption 3.3, $S < N$, so $S \bmod N = S$. $\square$

Corollary 4.5 (Vote Count Correctness). *Under Assumptions 3.4 and 3.5, the extracted vote count for candidate j equals the true tally:*

$$count_j = \sum_{i=1}^n v_{i,j}$$

Proof. By Theorem 4.4, $R = \sum x_i$. Each $x_i = \sum_j v_{i,j} \cdot 2^{b(k-j)}$, so:

$$R = \sum_i \sum_j v_{i,j} \cdot 2^{b(k-j)} = \sum_j \underbrace{\left(\sum_i v_{i,j} \right)}_{T_j} \cdot 2^{b(k-j)}$$

The slot extraction $\lfloor R/2^{b(k-j)} \rfloor \bmod 2^b$ isolates T_j , provided $T_j < 2^b$ (Assumption 3.5). Since each $v_{i,j} \in \{0, 1\}$, we have $T_j = \sum_i v_{i,j}$, which is the true vote count for candidate j . $\square$

5 Security Analysis

5.1 Cryptographic Assumption

The security of the Paillier cryptosystem rests on the **Decisional Composite Residuosity (DCR) assumption** [1]:

Definition 5.1 (DCR). *Given $N = pq$ and $z \in \mathbb{Z}_{N^2}^*$, it is computationally infeasible to distinguish whether z is an N -th residue modulo N^2 (i.e., $\exists y : z \equiv y^N \pmod{N^2}$) or a random element of $\mathbb{Z}_{N^2}^*$.*

Under DCR, the encryption $c_i = H^{sk_i} \cdot (1 + N)^{x_i} \bmod N^2$ is *semantically secure*: an adversary cannot distinguish c_i from a random element of $\mathbb{Z}_{N^2}^*$ without knowing the factorization of N .

5.2 Privacy Analysis

Proposition 5.2 (Individual Vote Privacy). *Under DCR, no computationally bounded adversary seeing c_i can learn x_i with non-negligible advantage, even given (N, H, pk_A) .*

Proof. $c_i = H^{sk_i} \cdot (1 + N)^{x_i} \bmod N^2$. The term H^{sk_i} is a random element of $\mathbb{Z}_{N^2}^*$ (since H is a generator and sk_i is random). The term $(1 + N)^{x_i}$ is the Paillier encryption of x_i . By the semantic security of Paillier under DCR [1], c_i is indistinguishable from random. $\square$

Proposition 5.3 (Aggregator Obliviousness). *The aggregator A , seeing $\{c_i\}_{i=1}^n$ and aux , learns only $S = \sum x_i$ and nothing about individual x_i values.*

Proof. A computes $R = S$ from the protocol. To learn x_i individually, A would need to decrypt c_i , which requires either the factorization of N (hard under the integer factorization assumption) or solving DCR. The value $aux = H^{sk_A \cdot \sum sk_i}$ is a single group element that encodes collective information; it does not reveal individual sk_i values under DCR. $\square$

Proposition 5.4 (Collector Obliviousness). *The collector C , seeing $\{aux_i\}_{i=1}^n$, learns nothing about any x_i .*

Proof. Each $aux_i = pk_A^{sk_i} = H^{sk_A \cdot sk_i} \bmod N^2$ is a random element of $\mathbb{Z}_{N^2}^*$ (since H is a generator and $sk_A \cdot sk_i$ is effectively random). Under DCR, these are indistinguishable from random. The collector sees no ciphertexts c_i and thus has no information about votes. $\square$

5.3 Integrity Properties

On-chain immutability. Each c_i is submitted as a Solana transaction. Once finalized, the on-chain record cannot be altered. This provides a verifiable audit trail: any discrepancy between the database and the blockchain can be detected.

Stage enforcement. The Solana program enforces election stages: votes are only accepted during the Voting stage, candidates only during Application. This prevents premature voting or late candidate addition.

RBAC. Sensitive operations require admin JWT. Voters cannot create elections or modify candidates.

5.4 Limitations

1. **No range proofs:** The system does not verify $v_{i,j} \in \{0, 1\}$. A malicious voter could submit a large value. Zero-knowledge range proofs would address this.
2. **Trusted setup:** TP knows the factorization of N . Distributed key generation would remove this trust.
3. **Single aggregator:** A knows sk_A and could fake results. Threshold decryption across multiple aggregators would mitigate this.
4. **No coercion resistance:** A voter can prove how they voted by revealing sk_i . Receipt-freeness mechanisms [3] would be needed.
5. **Local blockchain:** The test validator does not provide Byzantine fault tolerance. Mainnet deployment is needed for real security.

6 Vote Packing

To aggregate a multi-candidate election in a single Paillier ciphertext, votes are packed into a single integer using fixed-width bit slots. For k candidates with $b = 25$ bits per slot:

$$x_i = \sum_{j=1}^k v_{i,j} \cdot 2^{b(k-j)} = v_{i,1} \cdot 2^{b(k-1)} + v_{i,2} \cdot 2^{b(k-2)} + \dots + v_{i,k} \cdot 2^0 \quad (9)$$

Candidate 1 occupies the *most significant* slot (bits $b(k-1)$ to $bk-1$), and candidate k occupies the *least significant* slot (bits 0 to $b-1$). This reverse ordering is an implementation choice; the key requirement is consistency between packing and unpacking.

The homomorphic sum preserves slots independently because addition in $\mathbb{Z}$ is slot-wise when no slot overflows:

$$\sum_{i=1}^n x_i = \sum_{j=1}^k \left(\sum_{i=1}^n v_{i,j} \right) \cdot 2^{b(k-j)} \quad (10)$$

Extraction: $\text{count}_j = \lfloor R/2^{b(k-j)} \rfloor \bmod 2^b$.

Figure 3 illustrates the layout for $k = 10$, $b = 25$.

6.1 Capacity Analysis

The system capacity is determined by Assumptions 3.4 and 3.5:

- **Max candidates:** $k_{\max} = \lfloor \log_2 N/b \rfloor = \lfloor 255/25 \rfloor = 10$
- **Max votes per candidate:** $2^b - 1 = 33,554,431$
- **Max total voters:** $k_{\max} \times (2^b - 1) \approx 335.5$ million

For more candidates, a larger N is needed. Table 1 shows scaling requirements.

Table 1: Paillier modulus scaling for candidate count

Candidates (k)	Bits needed ($k \times 25$)	Min N bits	Min prime size
10	250	255	128-bit
20	500	505	253-bit
50	1,250	1,255	628-bit
100	2,500	2,505	1,253-bit

7 Algorithms

8 Evaluation

8.1 Test Methodology

We evaluated the system on a 16-core AMD Ryzen 9 with 32GB RAM, running Ubuntu 22.04 and Docker. The test script uses parallel encryption across all 16 cores and PostgreSQL COPY for bulk insertion.

8.2 Correctness Results

Table 2 shows results at various scales. In all cases, aggregated vote counts exactly matched expected values.

Algorithm 1 Voter Encryption (client-side, WASM)

Input: Vote vector $\mathbf{v} = [v_1, \dots, v_k] \in \{0, 1\}^k$, public params (N, H) , pk_A

Output: Ciphertext c , auxiliary value aux

Assumptions: 3.4 ($k \cdot 25 < \log_2 N$), 3.5 ($v_j \in \{0, 1\}$), 3.6 ($sk = r$)

```
1:  $sk \xleftarrow{\$} [2, N - 1]$  ▷ Generate secret key = randomness
2:  $m \leftarrow 0$ 
3: for  $j = 1$  to  $k$  do
4:    $m \leftarrow m + v_j \cdot 2^{25(k-j)}$  ▷ Pack in reverse order
5: end for
6:  $c \leftarrow \text{MODEXP}(H, sk, N^2) \cdot \text{MODEXP}(1 + N, m, N^2) \bmod N^2$ 
7:  $aux \leftarrow \text{MODEXP}(pk_A, sk, N^2)$ 
8: return  $(c, aux)$ 
```

Table 2: Stress test results. Encrypt includes parallel Paillier encryption and bulk DB insert. Agg is the libtommath aggregation time.

Voters	Candidates	Encrypt (s)	Agg (s)	DB Size	Correct
100	10	0.1	0.01	39 KB	✓
1,000	5	0.4	0.01	394 KB	✓
5,000	5	2.7	0.10	1.9 MB	✓

8.3 Performance Characteristics

Encryption throughput. Approximately 2,000–3,000 votes/second with 16 parallel workers. Bottleneck: modular exponentiation on 510-bit numbers.

Aggregation latency. Sub-second for 5,000 voters. Scales as $O(n)$ for the product step (one `mp_mul` per ciphertext) and $O(1)$ for the decryption step (fixed-size exponentiation regardless of n).

Capacity. 33.5 million voters ($10 \text{ candidates} \times 2^{25} - 1$ per slot), limited by Assumption 3.4.

9 Proof-of-Concept Implementation

The proposed architecture has been implemented as a proof-of-concept to validate the protocol. We briefly describe the key components; full source code is available at <https://github.com/Akshit11318/narad>.

9.1 Technology Stack

- **WASM:** C compiled with Emscripten 4.0.6, linking libtommath
- **Backend:** Node.js 20, Express, Prisma ORM
- **Native addon:** C + Node-API, cmake-js build, libtommath
- **Blockchain:** Solana (Anchor 0.30.1), test validator
- **Database:** PostgreSQL 16
- **Frontend:** React 19, Tailwind CSS
- **Deployment:** Docker Compose (4 containers)

Algorithm 2 Vote Aggregation (server-side, native libtommath)

Input: Ciphertexts $\{c_1, \dots, c_n\}$, aux , params (N, H, sk_A) , candidate count k

Output: Vote counts $[\text{count}_1, \dots, \text{count}_k]$

Assumptions: 3.1 ($N = pq$, odd), 3.2 ($\gcd(sk_A, N) = 1$), 3.3 ($S < N$)

```
1:  $N^2 \leftarrow N \cdot N$ 
2:  $sk_A^{-1} \leftarrow \text{MODINV}(sk_A, N)$  ▷ Exists by Assumption 3.2
▷ Step 1: Product of ciphertexts

3:  $\Pi \leftarrow 1$ 
4: for  $i = 1$  to  $n$  do
5:    $\Pi \leftarrow \Pi \cdot c_i \bmod N^2$  ▷ mp_mul + mp_mod
6: end for ▷ Step 2: Raise to  $sk_A$ 
▷ mp_exptmod
7:  $P \leftarrow \text{MODEXP}(\Pi, sk_A, N^2)$  ▷ Step 3: Cancel mask
▷ Exists by Assumption 3.1
8:  $aux^{-1} \leftarrow \text{MODINV}(aux, N^2)$ 
9:  $P' \leftarrow P \cdot aux^{-1} \bmod N^2$  ▷ Step 4: L-function
▷ Exact integer division
10:  $L \leftarrow (P' - 1) \text{div } N$  ▷ Step 5: Recover sum
▷ Now  $R = S = \sum x_i$ 
11:  $R \leftarrow L \cdot sk_A^{-1} \bmod N$  ▷ Step 6: Unpack votes
12: for  $j = 1$  to  $k$  do
13:    $\text{count}_j \leftarrow (R \gg 25(k - j)) \& (2^{25} - 1)$  ▷ Bit shift + mask
14: end for
15: return  $[\text{count}_1, \dots, \text{count}_k]$ 
```

9.2 Engineering Challenges Resolved

During implementation, seven critical bugs were identified and fixed:

1. **Even modulus:** Hardcoded N was even ($N \bmod 2 = 0$), breaking modular inverses. Fixed by generating proper semiprime $N = pq$.
2. **Degenerate key:** $sk_A = H$ produced trivial decryption. Fixed with separate random sk_A .
3. **Parameter source mismatch:** Aggregation read N from blockchain (stale) instead of database (current). Fixed by using database as single source of truth.
4. **Bit-packing mismatch:** C used 25-bit slots, JS fallback used 32-bit. Unified to 25-bit.
5. **Missing sk_A^{-1} step:** JS computed $L(P')$ but not $L(P') \cdot sk_A^{-1}$. Added the missing step.
6. **C GCD pre-check bug:** Faulty GCD check rejected valid sk_A . Removed pre-check; `mp_invmod` handles errors.
7. **Byte-order mismatch:** C binding produced big-endian bytes; codebase expected little-endian. Added byte reversal.

These bugs illustrate the gap between protocol design and implementation—particularly the subtlety of byte ordering in native bindings and the importance of a single source of truth for parameters.

9.3 Docker Deployment

All services run in Docker containers on a shared bridge network:

- `narad-postgres`: PostgreSQL 16 Alpine
- `narad-solana`: Solana test validator (`tchambard/solana-test-validator`)
- `narad-backend`: Node.js with native aggregator addon
- `narad-portal`: nginx serving React build

The backend auto-generates a Solana wallet on first startup and airdrops 100 SOL. The native aggregator is built inside the Docker image using `cmake-js`.

10 Related Work

Helios [2] is a web-based open-audit voting system using homomorphic encryption. It relies on a trusted authority for decryption and does not use blockchain.

Civitas [3] extends Helios with coercion resistance using distributed plaintext equivalence tests. NARAD does not address coercion resistance.

Aggregator-oblivious encryption [4] describes the protocol where users encrypt without key coordination. NARAD implements this protocol end-to-end.

Privacy-preserving aggregation [5] surveys aggregation protocols with formal privacy guarantees. NARAD’s protocol is an instance of this family.

11 Conclusion

We proposed NARAD, a microservices architecture for privacy-preserving blockchain voting based on Paillier homomorphic encryption. The key insight is that voters use their self-generated secret key as both the encryption randomness and the auxiliary value exponent, enabling mask cancellation without key coordination. We proved correctness under six explicit assumptions and analyzed security under DCR. A proof-of-concept implementation confirmed exact vote recovery at scale.

Future work includes: (1) zero-knowledge range proofs for vote validity, (2) threshold decryption across multiple aggregators, (3) distributed key generation to eliminate trusted setup, and (4) Solana mainnet deployment with program audits.

References

- [1] P. Paillier, “Public-key cryptosystems based on composite degree residuosity classes,” in *Proc. EUROCRYPT*, 1999, pp. 223–238.
- [2] B. Adida, “Helios: Web-based open-audit voting,” in *Proc. USENIX Security Symposium*, 2008, pp. 335–348.
- [3] A. Kiayias, M. Korman, and D. Walluck, “Voting without self-enforcing protocols: A survey of e-voting schemes,” in *Proc. ACNS*, 2008.
- [4] “Aggregator-oblivious encryption,” *IACR Cryptology ePrint Archive*, 2014. [Online]. Available: <https://eprint.iacr.org/2014/>

- [5] “Privacy-preserving aggregation protocols,” *Proceedings on Privacy Enhancing Technologies (PoPETs)*, vol. 2023, no. 1, 2023.
- [6] “libtommath: A portable number theoretic multiple-precision integer library,” [Online]. Available: <https://github.com/libtom/libtommath>
- [7] “Emscripten: LLVM to WebAssembly compiler,” [Online]. Available: <https://emscripten.org>
- [8] “Anchor: Solana Sealevel Framework,” [Online]. Available: <https://github.com/coral-xyz/anchor>
- [9] “Solana: A fast, secure, and censorship-resistant blockchain,” [Online]. Available: <https://solana.com>
- [10] “Docker: Containerized application platform,” [Online]. Available: <https://docker.com>

Protocol Execution Flow: Voter → Aggregation

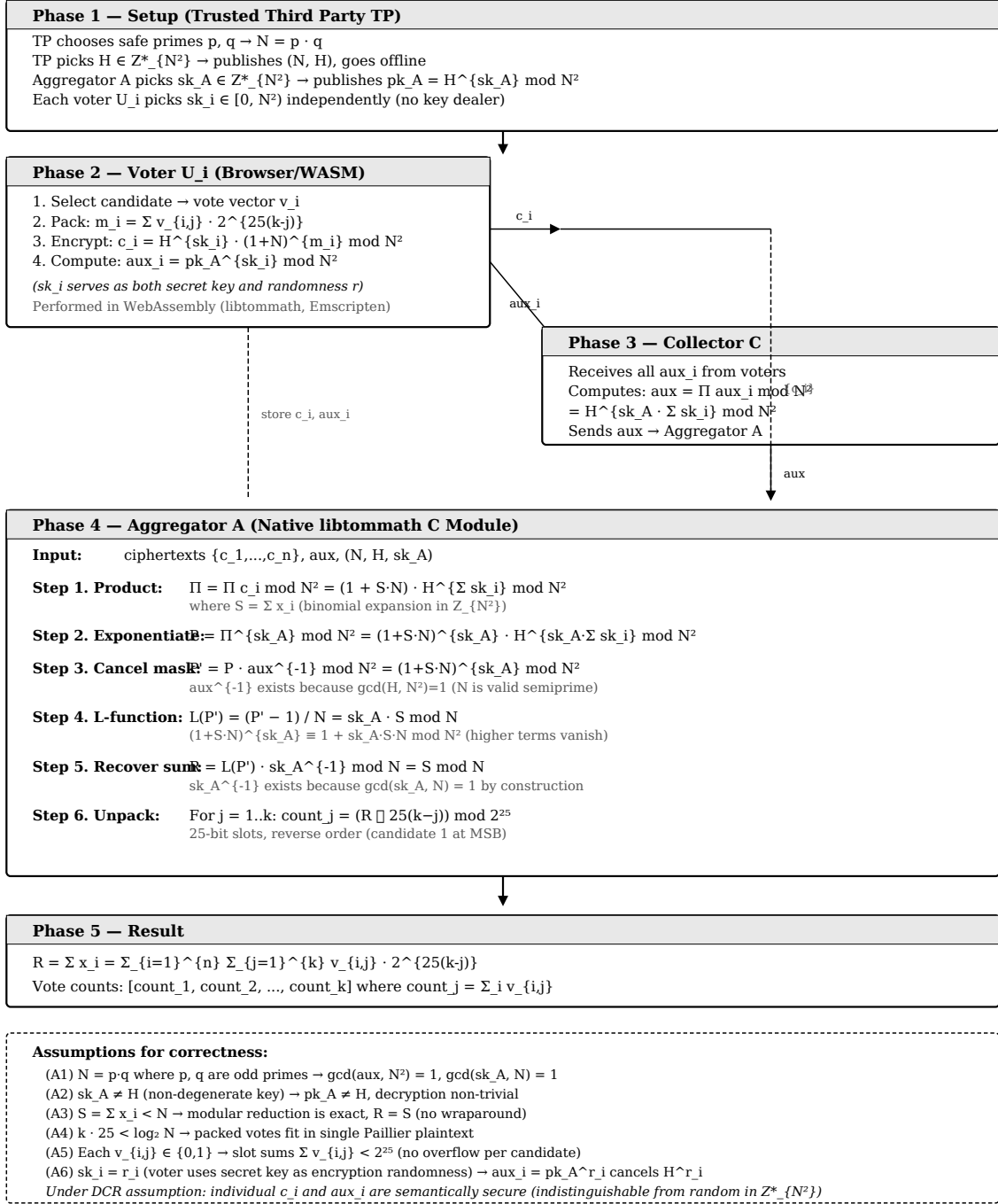


Figure 2: Protocol execution flow from voter to result. Phases 1–5 show the complete lifecycle. The assumptions box at the bottom lists all six conditions required for correctness.

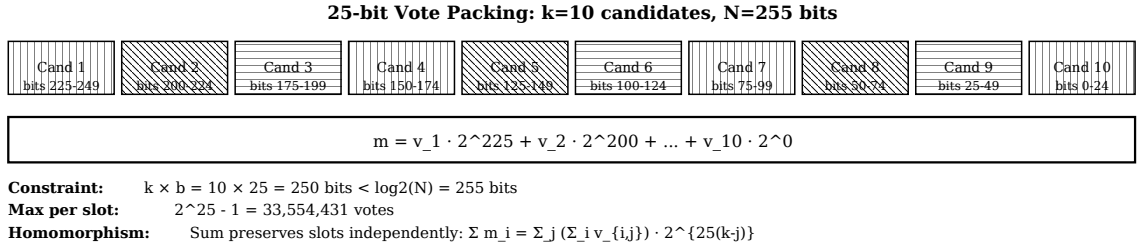


Figure 3: 25-bit vote packing for 10 candidates. Each slot occupies 25 bits. Total: $10 \times 25 = 250$ bits $< 255 = \lceil \log_2 N \rceil$. Maximum votes per candidate: $2^{25} - 1 = 33,554,431$.